

Replication and Cross-Domain Extension of Collaborative Filtering Models with Full-Catalog Ranking Evaluation

Muhil Ramesh and Tejas Hegde

05/03/2026

1 Project Goal

The goal of this project was to replicate and extend the main findings from Rajesh and Kumar’s paper, *Collaborative Filtering Models: An Experimental and Detailed Comparative Study*. The original paper compares several collaborative filtering models and evaluates both rating prediction accuracy and top- k recommendation quality.

For our replication, we focused on the MovieLens 1M dataset and implemented a representative subset of collaborative filtering models: KNNBasic Item-Item, SVD, NCF, LightGCN, and a Popularity baseline. Our goal was not to reproduce every experiment from the paper, but to test one of its main conclusions: traditional methods such as KNN and SVD are strong for rating prediction, while neural and graph-based models can perform better for top- k recommendation tasks.

We also extended the project by adding a simple Popularity baseline, implementing full-catalog ranking evaluation, adding Coverage as an additional metric, reporting F1@10 in our full-ranking tables, performing parameter sensitivity analysis, re-implementing neural and graph-based models in PyTorch, and testing cross-domain performance on Last.fm HetRec 2011.

2 Datasets

We used MovieLens 1M as the main replication dataset. We also used Last.fm HetRec 2011 as an additional cross-domain dataset to test whether the same model behavior held outside the movie recommendation domain.

Dataset	Interactions	Users	Items	Domain	Sparsity
MovieLens 1M	1,000,209	6,040	3,706	Movies	0.9553
Last.fm HetRec 2011	71,355	1,859	2,823	Music	0.9864

Table 1: Dataset statistics used in the replication and extension.

MovieLens 1M contains explicit movie ratings on a 1 to 5 scale. Last.fm contains user-artist listening counts, which we converted into pseudo-ratings using log scaling. This allowed us to run the same full-ranking recommendation evaluation on a music recommendation dataset.

3 Models Implemented

We implemented and evaluated the following models:

Model	Category	Purpose
KNNBasic Item-Item	Memory-based collaborative filtering	Traditional nearest-neighbor baseline
SVD	Model-based matrix factorization	Rating prediction baseline
NCF	Neural collaborative filtering	Neural ranking model
LightGCN	Graph-based collaborative filtering	Graph neural recommender
Popularity	Non-personalized baseline	Additional simple baseline

Table 2: Models used in our replication and extension.

KNNBasic and SVD were implemented using the Surprise library. NCF and LightGCN were implemented using PyTorch. LightGCN was trained with Bayesian Personalized Ranking (BPR) loss, which encourages positive items to receive higher scores than sampled negative items.

4 Evaluation Setup

We used two evaluation styles.

The first was an observed-test-pair evaluation. In this setup, models were evaluated only on items that appeared in the test set. This was useful for checking that the basic models worked, but it made ranking scores artificially high because each user was only being ranked over a small candidate set.

The second and more important evaluation was full-catalog ranking. In this setup, for each user, the model ranked all unseen items, removed items already seen during training, and selected the top 10 recommendations. This is a stricter and more realistic recommendation setting because the model must find relevant items from the full item catalog.

The main metrics used were:

Metric	Meaning
RMSE	Rating prediction error
MAE	Average absolute rating prediction error
Precision@10	Fraction of top 10 recommended items that were relevant
Recall@10	Fraction of relevant items recovered in the top 10
F1@10	Harmonic mean of Precision@10 and Recall@10
NDCG@10	Ranking quality with more weight on relevant items near the top
Coverage	Fraction of catalog items recommended across users

Table 3: Evaluation metrics used in the project.

5 Baseline Rating Prediction Results

We first ran 5-fold evaluation for KNNBasic Item-Item and SVD. These results are mainly useful for rating prediction accuracy and initial baseline comparison.

Model	RMSE	MAE
KNNBasic Item-Item	0.9993	0.7803
SVD	0.8736	0.6857

Table 4: Initial 5-fold rating prediction results for KNNBasic Item-Item and SVD.

SVD performed better than KNNBasic on both RMSE and MAE. This supports the paper’s general result that model-based collaborative filtering methods can outperform simpler memory-based methods for rating prediction.

6 MovieLens 1M Full-Ranking Results

After implementing full-catalog ranking, the results changed significantly. The scores became lower, which is expected because each model now had to rank relevant items against thousands of possible unseen items.

Model	Precision@10	Recall@10	F1@10	NDCG@10
LightGCN Full Ranking	0.2387	0.1605	0.1920	0.2911
NCF Full Ranking	0.1693	0.1097	0.1331	0.1958
Popularity Full Ranking	0.1520	0.0907	0.1136	0.1767
SVD Full Ranking	0.0781	0.0433	0.0557	0.0917
KNNBasic Item-Item Full Ranking	0.0313	0.0074	0.0120	0.0260

Table 5: Full-catalog ranking results on MovieLens 1M.

These results better match the trend expected from the paper. LightGCN performed the best on all ranking metrics, followed by NCF. Traditional baselines such as SVD and KNNBasic performed much worse under the full-catalog ranking setup.

This result suggests that graph-based and neural collaborative filtering models are better suited for top- k recommendation tasks, while SVD remains more useful for explicit rating prediction. The Popularity baseline also performed surprisingly well compared to SVD and KNNBasic, which shows that popularity effects are strong in recommendation datasets and should be included as a simple baseline.

7 Cross-Domain Extension: Last.fm HetRec 2011

To extend beyond MovieLens, we also evaluated the same full-ranking setup on Last.fm HetRec 2011. This dataset is from the music recommendation domain and contains user-artist listening counts. Since the data is based on listening frequency rather than explicit 1 to 5 ratings, we converted listening counts into pseudo-ratings using log scaling.

Model	Precision@10	Recall@10	F1@10	NDCG@10
LightGCN Full Ranking	0.0252	0.0837	0.0387	0.0615
Popularity Full Ranking	0.0114	0.0390	0.0176	0.0266
NCF Full Ranking	0.0050	0.0183	0.0078	0.0117
SVD Full Ranking	0.0019	0.0057	0.0028	0.0036
KNNBasic Item-Item Full Ranking	0.0014	0.0047	0.0021	0.0033

Table 6: Full-catalog ranking results on Last.fm HetRec 2011.

The Last.fm results preserve the most important trend: LightGCN achieved the best NDCG@10 and Recall@10. This suggests that the graph-based model was not only effective for movie recommendation, but also transferred better than the other models to a music recommendation dataset.

The Last.fm results also show that popularity is a strong baseline in recommendation tasks. On Last.fm, the Popularity baseline outperformed NCF on NDCG@10, even though NCF was a personalized neural model. This suggests that in some domains, especially music, global popularity can explain a large part of user behavior.

8 Cross-Dataset Comparison

Model	MovieLens 1M NDCG@10	Last.fm NDCG@10
LightGCN Full Ranking	0.2911	0.0615
NCF Full Ranking	0.1958	0.0117
Popularity Full Ranking	0.1767	0.0266
SVD Full Ranking	0.0917	0.0036
KNNBasic Item-Item Full Ranking	0.0260	0.0033

Table 7: NDCG@10 comparison between MovieLens 1M and Last.fm HetRec 2011.

Across both datasets, LightGCN achieved the best NDCG@10. This strengthens the main conclusion that graph-based collaborative filtering methods are especially effective for top- k recommendation. However, the exact ordering of the middle models changed. On MovieLens, NCF beat Popularity, while on Last.fm, Popularity beat NCF. This shows that model performance can depend on the dataset and domain.

9 Parameter Tuning and Sensitivity Analysis

For the parameter tuning extension, we tested the sensitivity of KNNBasic to the number of neighbors k . The values tested were $k = 10, 20, 40$, and 80 .

k	RMSE	MAE	Precision@10	Recall@10	NDCG@10	Coverage
10	1.0345	0.8049	0.6409	0.6137	0.8116	0.7357
20	1.0080	0.7858	0.6529	0.6186	0.8234	0.7029
40	0.9993	0.7803	0.6582	0.6199	0.8252	0.6909
80	1.0019	0.7855	0.6559	0.6175	0.8189	0.7105

Table 8: KNNBasic sensitivity analysis over different neighborhood sizes.

The best KNN result was around $k = 40$. Increasing from 10 to 40 improved RMSE, Precision@10, Recall@10, and NDCG@10. However, increasing to 80 did not improve the result and slightly reduced ranking performance. This shows that KNN is sensitive to the neighborhood size, and that choosing too small or too large of a neighborhood can hurt performance.

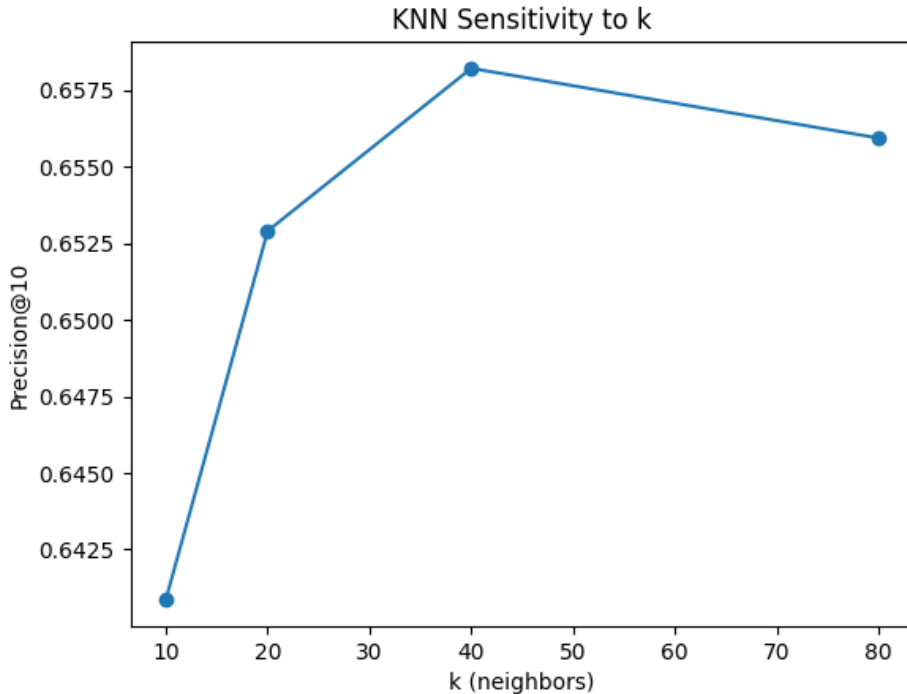


Figure 1: Sensitivity of KNNBasic Item-Item performance to the number of neighbors k . Precision@10 improves as k increases from 10 to 40, but slightly decreases at $k = 80$.

We also tested SVD with different latent factor values. The best RMSE occurred around 50 latent factors, while 100 factors performed slightly worse. This suggests that increasing model complexity does not always improve performance and may lead to worse generalization.

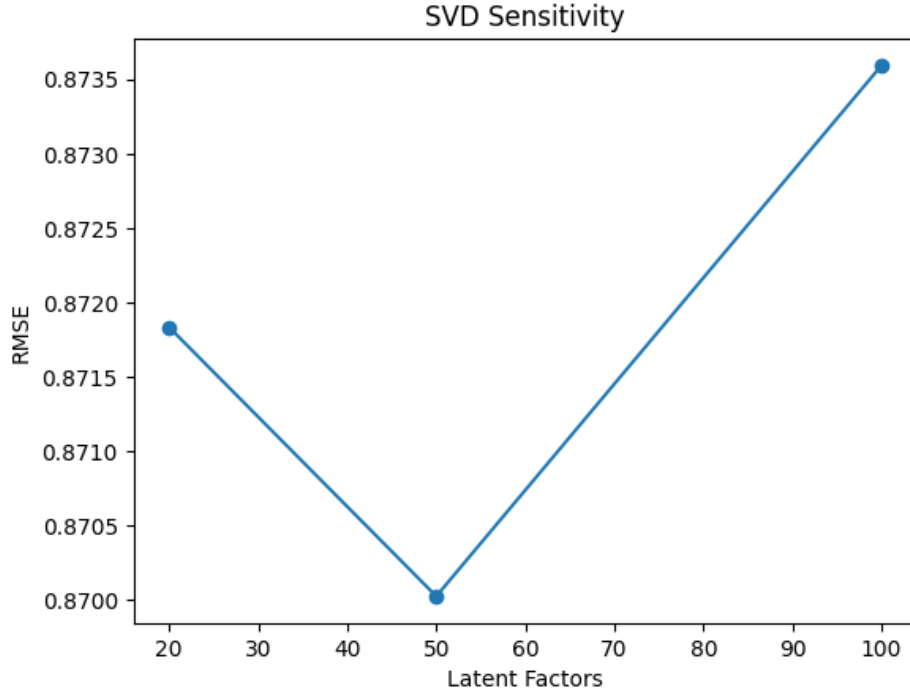


Figure 2: Sensitivity of SVD performance to the number of latent factors. The lowest RMSE occurs around 50 latent factors, suggesting that increasing model complexity does not always improve performance.

10 Extensions

Our project included five main extensions beyond simply rerunning the original paper’s experiments, plus one additional evaluation-protocol change.

Extension	What We Did
Extended to a new domain dataset	We evaluated the same models on Last.fm HetRec 2011 to test whether the results generalized beyond MovieLens.
Extended metrics	We included Coverage as an additional metric beyond the main accuracy and ranking metrics.
Parameter tuning / sensitivity analysis	We tested the effect of KNN neighborhood size and SVD latent factor size on model performance.
Added a new baseline algorithm	We added a non-personalized Popularity baseline and compared it against the personalized models under full-catalog ranking.
Re-implemented models	We implemented NCF and LightGCN in PyTorch instead of only relying on library implementations.
Additional evaluation-protocol change	We implemented full-catalog ranking so that models ranked all unseen items instead of only observed test items.

Table 9: Extensions included in our project.

11 Discussion

The most important result from our project is that the evaluation protocol strongly affects the conclusions. In the observed-test-pair setup, SVD appeared to outperform the neural and graph-based models on ranking metrics. However, after switching to full-catalog ranking, LightGCN became the top model, followed by NCF. This better supports the paper’s main conclusion that graph-based and neural methods are strong for top- k recommendation.

This also shows why recommender system evaluation needs to be carefully designed. Ranking only observed test items can inflate results and make models look stronger than they actually are. Full-catalog ranking is more difficult because the model must find relevant items from the entire item catalog.

The cross-domain Last.fm results also support the importance of dataset choice. LightGCN remained the strongest model, but the relative order of NCF and Popularity changed. On MovieLens, NCF beat Popularity, but on Last.fm, Popularity beat NCF. This suggests that some datasets have stronger popularity effects than others, and that recommender performance depends on both the model and the domain.

Another important finding is that model complexity involves trade-offs. LightGCN performed best for full-ranking recommendation, but it required custom training, graph construction, negative sampling, and BPR optimization. SVD was easier to train and performed well on RMSE and MAE, but it did not perform as well for full-catalog ranking. KNNBasic was the simplest collaborative filtering model in our experiment, but it had the weakest full-ranking performance.

The parameter tuning results also showed that larger models or larger hyperparameter values do not always improve performance. For KNNBasic, $k = 40$ performed best, while increasing to $k = 80$ slightly hurt ranking performance. For SVD, the best RMSE occurred around 50 latent factors, while 100 factors performed worse. This supports the idea that recommender systems require careful hyperparameter tuning instead of simply increasing model size.

12 Conclusion

Our replication supports the paper’s broader conclusion that neural and graph-based collaborative filtering methods can outperform traditional methods for top- k recommendation. In our full-ranking evaluation on MovieLens 1M, LightGCN achieved the strongest ranking performance, followed by NCF. SVD performed better than KNNBasic for rating prediction, but it was weaker under full-catalog ranking.

Our cross-domain extension on Last.fm also showed that LightGCN remained the strongest model by NDCG@10, suggesting that graph-based collaborative filtering can generalize beyond the MovieLens movie recommendation setting. At the same time, the Last.fm results showed that simple popularity baselines can still be very competitive in some domains.

Overall, our results show that the best model depends on the evaluation goal and the dataset. SVD is useful for rating prediction, while LightGCN is more effective for personalized top- k recommendation. Our extensions also show that simple baselines, ranking protocol choices, hyperparameter sensitivity, and dataset domain all play an important role in understanding recommender system performance.

References

- [1] Rajesh Devangam Bangaru, and Avadhesh Kumar. *Collaborative Filtering Models: An Experimental and Detailed Comparative Study*. Scientific Reports, vol. 15, no. 1, 2025. DOI: <https://doi.org/10.1038/s41598-025-15096-4>